

DESIGN PATTERNS CASE STUDY

Healthcare Doctor Appointment Management System

AI-Assisted Discovery, Architectural Evolution & 4-Language Empirical Analysis

Course: 23CSE455 — Software Design Patterns | **Team:** B2

Original Repository: [arnobt78/HealthCare-Doctor-Appointment-Management-System](#) (Commit f846cd4)

Case Study Repository: [Karthek-3/B2](#) | Final Deliverables: Part 1, Part 2 & Part 3

Team Members: [Karthek](#), [Aswathy Mohan](#), [Arnob](#), et al.

Case Study Problem Statement & Project Objectives

HEALTHCARE DOMAIN CHALLENGES

- Procedural Form Logic: Inline switch statements handling create, schedule, and cancel workflows.
- Unconstrained Scheduling: Hardcoded date pickers lacking shift limits or rest buffers.
- Direct Notification Coupling: Mutations triggering inline SMS calls without queueing or retries.
- Vendor Lock-In: Backend logic directly tied to node-appwrite SDK client functions.

DESIGN PATTERN ARCHITECTURE

- Encapsulation & OCP: Decouple domain rules from framework and vendor dependencies.
- Interchangeable Policies: Strategy pattern for outpatient, shift, and emergency scheduling.
- Vendor Independence: Adapter pattern for teleconsultation video conferencing APIs.
- Event-Driven Decoupling: Observer pattern for background reminder processing & audit logging.

CASE STUDY METHODOLOGY

- Part 1: AI-assisted discovery & empirical classification of original baseline repository.
- Part 2: Evolution implementation across 3 core healthcare feature enhancements.
- Part 3: 4-Language implementation (Java, Python, JS, C++) across 7 design patterns.
- Benchmarking: Reproducible execution testing & Lizard complexity analysis.

Original Application Architecture & Technology Stack

TECHNOLOGY STACK & INFRASTRUCTURE

- **Framework:** Next.js (App Router, Server Actions)
- **UI Layer:** React 18, Tailwind CSS, Radix UI
- **Validation:** Zod Schema Validation Engine
- **Backend BaaS:** Appwrite (Databases, Users, Messaging, Storage)
- **Messaging:** Twilio SMS API integration
- **Baseline Commit:** f846cd4c476d48a04fcaeb530b48a77c5295bb92

BASELINE APPLICATION WORKFLOW

- **1. Patient Onboarding:** Form submission via AppointmentForm.tsx
- **2. Schema Validation:** Parameterized validation in validation.ts
- **3. Action Processing:** Server action routines in appointment.actions.ts
- **4. Database Mutation:** Direct appwrite.databases.createDocument call
- **5. Notification Side-Effect:** Inline call to sendSMSNotification helper
- **6. Result Rendering:** Client UI state update on success screen

Part 1: AI-Assisted Pattern Discovery & Classification

EMPIRICAL DISCOVERY INVENTORY SUMMARY

0 Classical GoF Patterns Fully Implemented Baseline | 2 Pattern-Like Structures | 5 Refactoring Opportunities

PATTERN-LIKE STRUCTURES (2)

1. Singleton (P01): Module-scoped Appwrite client instantiated once in `lib/appwrite.config.ts:29-40`. Reused across requests, missing DCL/private constructor.
2. Factory Method (P02): Parameterized `getAppointmentSchema` switch function in `lib/validation.ts:148-157` returning validation schemas.

REFACTORING OPPORTUNITIES (5)

1. Strategy (P04): Inline switch branching in `AppointmentForm.tsx:58-115`.
2. Adapter (P07): Direct node-appwrite SDK coupling in `lib/appwrite.config.ts`.
3. Observer (P03): Procedural SMS call in `appointment.actions.ts:134-165`.
4. State (P05): Raw union status strings ('pending' | 'scheduled').
5. Command (P06): Async server functions without execute/undo abstractions.

Part 1: Source-Code Evidence & False-Positive Elimination

HUMAN SOURCE VERIFICATION VS LLM HALLUCINATIONS

- **The AI Hallucination Trap:** LLM tools initially tagged procedural switch statements as GoF Strategy patterns and raw Appwrite SDK initialization as formal Singleton patterns.
- **Empirical Code Auditing:** Inspecting actual line numbers (e.g. AppointmentForm.tsx:58-115) proved the total absence of GoF interfaces, abstract classes, or polymorphic dispatch.
- **Pattern-Like vs Classical GoF:** Recognized module-level client caching (lib/appwrite.config.ts:29-40) as pattern-like Singleton, and getAppointmentSchema as a parameterized factory function.
- **Refactoring Opportunity Baseline:** Formalized 5 key refactoring opportunities (Adapter, Strategy, Observer, State, Command) to serve as the baseline for Part 2 and Part 3 evolutionary design.

Part 2: Application Evolution & Pattern Architecture

CH01: VIDEO CONSULTATION

Pattern: GoF Adapter

- Goal: Add teleconsultation without vendor lock-in.
- Target Interface: IVideoConsultationService
- Adapter: LocalDemonstrationVideoAdapter
- Adaptee: ProprietaryVideoProvider
- Data Mapping: ISO-8601 <-> Unix epoch seconds, duration minutes <-> seconds, role LEVEL_HOST/LEVEL_GUEST.

CH02: DOCTOR AVAILABILITY

Pattern: GoF Strategy

- Goal: Dynamic scheduling rule evaluation.
- Strategy Interface: IAvailabilityStrategy
- Context: DoctorAvailabilityContext
- Policies: StandardBusinessHours, ShiftBasedAvailability, EmergencyOnCall.
- Rules: Lunch break (12-13), shift limits (07-15), 15-min emergency recovery buffer.

CH03: AUTOMATED REMINDERS

Pattern: GoF Observer

- Goal: Decouple mutations from SMS delivery.
- Subject: DomainEventBus
- Subscriber Interface: IDomainEventSubscriber
- Observers: ReminderSchedulerObserver, ReminderCancellationObserver, AuditLogObserver.
- Engine: Persistent JSON store & T-24h/T-2h reminder processing engine.

Part 2 Architecture: Before vs. After Pattern Refactoring

BEFORE EVOLUTION (PROCEDURAL)

- UI Form Coupling: Inline switch branching inside AppointmentForm.tsx handling all create/schedule actions.
- Hardcoded Availability: Date picker allowing arbitrary booking without shift or rest buffer validation.
- Procedural Notification: Appointment mutations directly calling sendSMSNotification inside server actions.
- Infrastructure Coupling: Direct calls to node-appwrite SDK client endpoints without abstraction.

AFTER EVOLUTION (GOF PATTERNS)

- Adapter (CH01): Decoupled video conferencing normalizing epoch timestamps, durations, and LEVEL_HOST roles.
- Strategy (CH02): Interchangeable availability policies (Standard, Shift, Emergency) evaluated at runtime via DoctorAvailabilityContext.
- Observer (CH03): Asynchronous DomainEventBus publishing APPOINTMENT_SCHEDULED to background reminder scheduler with JSON store persistence.

Part 2: Empirical Verification & Regression Testing Results

VERIFIED TEST SUITE OUTCOMES & BOUNDARY CHECKS

- **CH01 Adapter Tests (6/6 Passed):** Verified session creation, room naming (room_pat_1_doc_1), token role minting (LEVEL_HOST / LEVEL_GUEST), role rejection ('admin'), and DomainValidationError mapping.
- **CH02 Strategy Tests (6/6 Passed):** Verified standard business hours, weekend rejection, 12:00-13:00 lunch break overlap, shift limits (07:00-15:00), and 15-min emergency rest buffer enforcement.
- **CH03 Observer Tests (6/6 Passed):** Verified APPOINTMENT_SCHEDULED event publishing, subscriber dispatch, T-24h/T-2h reminder calculation, cancellation handling, retry policy, and JSON persistence.
- **Zero Regression Guarantee:** Verified 100% backward compatibility for original patient registration, appointment booking, and status updates across the Next.js server action layer.

Part 3: Cross-Language Implementation Architecture (4 Ecosystems)

JAVA 17/25

- Static Typing & OOP
- Package scoping (snippet)
- Interfaces & Abstract Classes
- HashMap repository
- Double-Checked Locking Singleton
- Ex: `java -cp Part3/strategy/java/bin test.StrategyTest`

PYTHON 3.14

- Dynamic Typing & ABC
- `abc.ABC` & `@dataclass`
- Dictionary repository
- Thread-safe `__new__` Singleton
- Self-contained `sys.path` runner
- Ex: `python Part3/strategy/python/tests/test_strategy.py`

JAVASCRIPT (NODE)

- Prototypal / ES6 Classes
- CommonJS module export
- Map collection repository
- Class property Singleton
- Dynamic duck-typing checks
- Ex: `node Part3/strategy/javascript/tests/test_strategy.js`

C++14 / GCC (WSL)

- Statically Typed & Native
- Pure virtual functions
- `std::unordered_map`
- `std::unique_ptr` & RAII
- `std::lock_guard` Singleton
- Ex: `wsl -e bash -c "./test_strategy"`

Part 3: Empirical Dataset & Benchmark Metrics (28 Implementations)

CANONICAL DATASET METRICS SUMMARY (Part3/llm/pattern_crosslanguage.csv)

- **100% Test Pass Rate:** All 28 pattern-language test suites (7 patterns x 4 languages) executed and passed 100% of test cases (28/28 verified passing).
- **Source Code Volume (LOC):** Python achieves highest conciseness (22-85 LOC/pattern), followed by JS (20-109 LOC), C++ (42-141 LOC), and Java (32-152 LOC).
- **Max Cyclomatic Complexity:** Lizard max function CC (max_function_cc) ranges from 1 to 12 across all implementations (e.g. State CC=1, Strategy CC=12).
- **Verified Toolchains:** Java: OpenJDK 25.0.4.1 | Python: 3.14.8 | JS: Node.js v24.21.0 | C++: GCC 15.2.0 (WSL Ubuntu).
- **Process Median Ms (5 Runs):** Python (26-61ms) & Node.js (36-53ms) show fast process startup; Java averages 52-96ms; C++ in WSL (97-118ms) includes Linux subsystem process launch overhead.

Discussion, Architectural Trade-Offs & Key Takeaways

LESSONS LEARNED & ARCHITECTURAL IMPACT

- **Decoupling vs Indirection Overhead:** Introducing Strategy, Adapter, and Observer increases interface count but eliminates monolithic switch statements and direct vendor lock-in, enabling isolated unit testing.
- **Cross-Language Adaptability:** Core GoF design pattern principles (encapsulation, polymorphism, event buses) translate cleanly across both statically typed (Java, C++) and dynamically typed (Python, JS) languages.
- **AI-Assisted Software Engineering:** LLMs significantly accelerate pattern refactoring and boilerplate generation, but automated test suites and human code audits are mandatory to eliminate subtle hallucinations.
- **Final Case Study Conclusion:** The case study successfully demonstrates pattern discovery in Next.js, 3-scenario architectural evolution, and complete 4-language empirical evaluation.

Team Contributions, Repository References & Q&A

TEAM B2 CONTRIBUTIONS

- **Karthek:** Part 3 Command & State implementations, C++ WSL toolchain integration, cross-language benchmark suite.
- **Aswathy Mohan:** Part 2 Evolution scenarios (Adapter, Strategy, Observer), availability engine, test suite design.
- **Arnob:** Baseline Next.js repository analysis, Part 1 pattern discovery & empirical code auditing.
- **Team B2:** CSV dataset consolidation, slide deck creation, PDF export verification, and final submission audit.

REPOSITORIES & REFERENCES

- **Team Case Study Repo:** <https://github.com/Karthek-3/B2>
- **Original Application:** arnobt78/HealthCare-Doctor-Appointment-Management-System--NextJS-FullStack
- **Baseline Commit:** f846cd4c476d48a04fcaeb530b48a77c5295bb92
- **Assignment Rubric:** DP_Csestudy(1).pdf & DP_Csestudy(1)(1).pdf
- **Canonical Dataset:** Part3/llm/pattern_crosslanguage.csv

THANK YOU! QUESTIONS & ANSWERS